📝 Практическое задание: EDA и бинарная классификация на датасете «Титаник»

Дисциплина: Машинное обучение

Модуль 1: Введение в Data Science и EDA

Форма сдачи: Jupyter Notebook (`.ipynb`) + репозиторий на GitHub

Максимальный балл: 100

Срок сдачи: 2 недели с момента выдачи

Учебный год: 2025/2026

📋 Структура отчёта

Ваш отчёт должен быть оформлен как **Jupyter Notebook** (`.ipynb`) и содержать следующие **14 разделов** (каждый раздел — отдельная ячейка Markdown + соответствующие ячейки кода):

#	Раздел	Формат	Что внутри
1	**Титульный лист**	Markdown	Название работы, ФИО, группа, ссылка на репозиторий, дата
2	**Введение**	Markdown	Цель работы, постановка задачи, актуальность
3	**Теоретическая часть**	Markdown	Ключевые понятия (EDA, пропуски, кодирование, масштабирование, метрики)
4	**Описание данных**	Markdown + код	Источник, статистика, структура датасета, распределение классов
5	**Подготовка данных и EDA**	Код + Markdown	Загрузка, визуализация (≥5 графиков), обработка пропусков, Feature Engineering, кодирование
6	**Построение и обучение моделей**	Код + Markdown	Logistic Regression + Decision Tree, время обучения, обоснование
7	**Оценка качества**	Код + Markdown	Accuracy, Precision, Recall, F1, ROC-AUC, матрица ошибок, ROC-кривая

#	Раздел	Формат	Что внутри
8	**Интерпретация результатов**	Код + Markdown	Коэффициенты LR, Feature Importance DT, бизнес-инсайты
9	**Функция предсказания**	Код + Markdown	`predict_passenger()`, 5+ демонстрационных примеров
10	**Сохранение модели**	Код + Markdown	Сохранение в `.pkl` через `joblib`, метаданные (`.json`)
11	**Загрузка модели из репозитория**	Код + Markdown	Загрузка через raw-ссылку, проверка работоспособности
12	**Выводы**	Markdown	Что получилось / трудности / идеи улучшения
13	**Список источников**	Markdown	Минимум 5 источников (книги, статьи, документация)
14	**Приложение**	Markdown	Полный код с комментариями, `requirements.txt`

📌 **Требования к оформлению ячеек:**

- Все разделы **начинаются с ячейки Markdown** с заголовком `##` (второй уровень)
- Каждый раздел — **отдельный блок** ячеек (Markdown → Код → Markdown с выводами)
- Комментарии в коде — **обязательны**, минимум 1 комментарий на 2–3 строки
- Порядок разделов — **строго по таблице выше**, без пропусков

🗀 Структура единого репозитория на GitHub

⚠ **ВАЖНО:** Все модули курса хранятся **в одном репозитории** — `ml-course-{фамилия}`. Каждый модуль — в отдельной папке.

```
ml-course-ivanov/                          ← ОДИН репозиторий на весь курс
│
├── README.md                              ← Общее описание курса + ссылки на все модули
├── .gitignore
│
├── module-1-titanic/                      ← 🗀 Модуль 1 (текущее задание)
```

```
│   ├── README.md                          ← Описание именно Модуля 1
│   ├── notebook.ipynb                     ← Полный отчёт (14 разделов)
│   ├── data/
│   │   ├── train.csv
│   │   ├── test.csv
│   │   └── titanic_info.md                ← Описание датасета
│   ├── models/
│   │   ├── lr_model.pkl
│   │   ├── dt_model.pkl
│   │   ├── scaler.pkl
│   │   ├── le_sex.pkl
│   │   ├── feature_cols.json
│   │   └── metrics.json
│   ├── examples/
│   │   ├── eda_plots.png
│   │   ├── roc_curves.png
│   │   └── confusion_matrices.png
│   └── requirements.txt
│
├── module-2-{название}/                   ← 🗀 Модуль 2 (будет позже)
│   ├── README.md
│   ├── notebook.ipynb
│   ├── data/
│   ├── models/
│   └── requirements.txt
│
├── module-3-{название}/                   ← 🗀 Модуль 3 (будет позже)
│   └── ...
│
├── module-4-{название}/                   ← 🗀 Модуль 4 (будет позже)
├── module-5-{название}/                   ← 🗀 Модуль 5 (будет позже)
├── module-6-{название}/                   ← 🗀 Модуль 6 (будет позже)
└── module-7-{название}/                   ← 🗀 Модуль 7 (будет позже)
```

📌 **Правила именования:**

- **Репозиторий:** `ml-course-{фамилия}` (например, `ml-course-ivanov`)
- **Папки модулей:** `module-{N}-{краткое-название}` (например, `module-1-titanic`, `module-2-regression`)
- **Файлы внутри модуля:** одинаковые для всех модулей (`notebook.ipynb`, `README.md`, `data/`, `models/`)

📌 README.md в корне репозитория (обязательно):

```
# 🎓 ML Course — Портфолио проектов

**Студент:** Иванов Иван Иванович
**Группа:** ПКТб-23-1
**Дисциплина:** Машинное обучение и ИИ
**Учебный год:** 2025/2026

## 📚 Модули курса

| # | Модуль | Тема | Статус | Ссылка |
|---|--------|------|--------|--------|
| 1 | module-1-titanic | EDA + бинарная классификация | ☑ Готово |
[Открыть](./module-1-titanic/) |
| 2 | module-2-... | ... | ⏳ В работе | — |
| 3 | module-3-... | ... | ⏳ Ожидается | — |
| 4 | module-4-... | ... | ⏳ Ожидается | — |
| 5 | module-5-... | ... | ⏳ Ожидается | — |
| 6 | module-6-... | ... | ⏳ Ожидается | — |
| 7 | module-7-... | ... | ⏳ Ожидается | — |
```

🎯 Цель работы

Освоить **полный цикл ML-проекта** на примере задачи бинарной классификации: от загрузки сырых данных и разведочного анализа (EDA) до обучения моделей, оценки качества, интерпретации результатов и создания интерактивного предсказания.

📦 Что нужно сдать

Артефакт	Формат	Куда
Jupyter Notebook с отчётом	`.ipynb`	`ml-course-{фамилия}/module-1-titanic/`
Обученная модель	`.pkl` (joblib)	`ml-course-{фамилия}/module-1-titanic/models/`
Список признаков	`.json`	`ml-course-{фамилия}/module-1-titanic/models/`

Артефакт	Формат	Куда
Метрики	`.json`	`ml-course-{фамилия}/module-1-titanic/models/`
README.md модуля	`.md`	`ml-course-{фамилия}/module-1-titanic/`
README.md репозитория	`.md`	`ml-course-{фамилия}/`
Ссылка на репозиторий	URL	В титульном листе ноутбука

📊 Выбор датасета

Базовый датасет (обязательный): Titanic — Machine Learning from Disaster

🔗 Ссылка: https://www.kaggle.com/c/titanic

На странице соревнования:

1. Нажать **«Join Competition»** (требуется регистрация)
2. Принять правила
3. Скачать `train.csv` (891 строка) и `test.csv` (418 строк)

Альтернативный датасет (для тех, кто хочет +баллы):

- **Spaceship Titanic** — https://www.kaggle.com/c/spaceship-titanic (более сложный, с большим количеством пропусков и категориальных признаков)

⚠ **Важно:** если берёте Spaceship Titanic — обязательно обоснуйте выбор в разделе «Введение» и адаптируйте все шаги под его структуру.

📋 Пошаговые инструкции по каждому разделу отчёта

1️⃣ Титульный лист

Что сделать:

Создайте **первую ячейку Markdown** в ноутбуке с заголовком первого уровня (`#`) и следующим содержимым:

```
# EDA и бинарная классификация на датасете «Титаник»

**Дисциплина:** Машинное обучение и ИИ
**Модуль 1:** Введение в Data Science и EDA
**Датасет:** Titanic (или Spaceship Titanic)

**Выполнил:** Иванов Иван Иванович
```

```
**Группа:** ПКТб-23-1
**Преподаватель:** [ФИО]

**Ссылка на репозиторий:** https://github.com/username/ml-course-ivanov
**Путь к модулю:** `module-1-titanic/notebook.ipynb`
**Дата сдачи:** 2026-XX-XX
```

Требования:

- ☑ Использовать форматирование Markdown (`**жирный**`, `*курсив*`)
- ☑ Ссылка на репозиторий должна быть **рабочей** (открывается в браузере)
- ☑ **Обязательно** указать путь к модулю внутри репозитория
- ☑ Указать точное название выбранного датасета

Баллы: 3

---

2 Введение

Что сделать:

Создайте ячейку Markdown с заголовком `## Введение`. Ответьте на вопросы:

1. **Какую задачу решаем?** (бинарная классификация: выжил / не выжил)
2. **Что такое ML-проект?** (2–3 предложения своими словами о жизненном цикле)
3. **Почему это классификация, а не регрессия?** (объясните разницу)
4. **Какая метрика важнее в этой задаче?** (Accuracy vs F1 vs ROC-AUC — обоснуйте)
5. **Цель работы** — сформулируйте 3 конкретных пункта:
    - «Провести полный EDA датасета Титаник»
    - «Обучить 2+ модели и достичь ROC-AUC ≥ 0.80»
    - «Реализовать функцию предсказания для нового пассажира»

Пример структуры:

```
## Введение

**Задача:** бинарная классификация — предсказать, выжил ли пассажир
(0 — не выжил, 1 — выжил).

**ML-проект** — это последовательность этапов: сбор данных → EDA →
предобработка → feature engineering → обучение → оценка → интерпретация.

**Почему классификация, а не регрессия:** ...
```

Баллы: 5

---

3 Теоретическая часть

Что сделать:

Создайте ячейку Markdown. **Своими словами** объясните 6 ключевых понятий:

Понятие	Что объяснить	Минимум
EDA (Exploratory Data Analysis)	Что это, зачем нужен, какие шаги	4–5 предложений
Пропуски (NaN)	Почему возникают, стратегии обработки	4–5 предложений
Кодирование категорий	Label Encoding vs One-Hot Encoding	3–4 предложения
Масштабирование	StandardScaler vs MinMaxScaler	3–4 предложения
Feature Engineering	Зачем создавать новые признаки	3–4 предложения
Метрики классификации	Accuracy, Precision, Recall, F1, ROC-AUC	5–6 предложений

Требования:

- ☑ Формулы для каждой метрики (LaTeX)
- ☑ Минимум **1 визуальный элемент** — таблица, схема или график
- ❌ Не копировать из Википедии — должно быть видно ваше понимание

Формулы (обязательно в LaTeX):

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP} \quad Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Баллы: 12

---

4 Описание данных

Что сделать:

1. Загрузите датасет **в отдельной ячейке кода**.
2. Создайте ячейку Markdown со следующими данными:

```
## Описание данных

**Источник:** [ссылка на Kaggle]

**Структура:**
```

```
- Всего примеров в train: 891
- Всего примеров в test: 418
- Признаков: 11 (+ целевая `Survived`)
- Числовых признаков: X
- Категориальных признаков: Y

**Описание столбцов:**
| Столбец | Тип | Описание | Пропуски |
|---------|-----|----------|----------|
| PassengerId | int64 | ID пассажира | 0% |
| Survived | int64 (0/1) | Целевая: выжил или нет | 0% |
| Pclass | int64 (1/2/3) | Класс билета | 0% |
| ... | ... | ... | ... |

**Распределение классов:**
- Не выжило (0): 549 (61.6%)
- Выжило (1): 342 (38.4%)

**Файл данных в репозитории:** `module-1-titanic/data/titanic_info.md`
```

Требования:

- ☑ **Обязательно** вывести в коде: `print(train_df.shape, test_df.shape)`
- ☑ Создать файл `data/titanic_info.md` внутри `module-1-titanic/`
- ☑ Указать долю каждого класса (дисбаланс)

Баллы: 6

---

5 Подготовка данных и EDA

Что сделать:

В ячейках кода выполнить:

1. **Загрузка датасета** (`pd.read_csv()`)
2. **Первичный осмотр**: `df.head()`, `df.info()`, `df.describe()`
3. **Анализ пропусков**: `df.isnull().sum()` + визуализация
4. **Статистический анализ**: `skew()`, `kurtosis()`, `value_counts()`
5. **Визуализация (минимум 5 графиков)**:
   - 📊 **Countplot** — распределение целевой переменной
   - 📈 **Гистограмма** — распределение Age (с `hue='Survived'`)
   - 📦 **Boxplot** — Age по Pclass и Survived
   - 🔥 **Heatmap** — корреляционная матрица числовых признаков
   - 📉 **Barplot** — выживаемость по Sex / Pclass / Embarked
6. **Обработка пропусков**:

- `Age` — медиана по группам (Pclass + Sex)
- `Embarked` — мода
- `Cabin` — удалить столбец

7. **Feature Engineering** — создать:
   - `Family_Size = SibSp + Parch + 1`
   - `Is_Alone = (Family_Size == 1).astype(int)`
   - `Age_Group` — биннинг через `pd.cut()`
8. **Кодирование**:
   - `Sex` → LabelEncoder
   - `Embarked` → One-Hot (`get_dummies`)

Пример кода:

```
# Загрузка
train_df = pd.read_csv('data/train.csv')  # ← путь внутри module-1-titanic/
test_df = pd.read_csv('data/test.csv')

# Первичный осмотр
print(train_df.shape)
print(train_df.head())
print(train_df.info())
print(train_df.describe())

# Анализ пропусков
missing = train_df.isnull().sum()
missing_pct = (missing / len(train_df)) * 100
# ... вывод таблицы

# Визуализация — сетка 2x3
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
sns.countplot(data=train_df, x='Survived', ax=axes[0,0])
sns.histplot(data=train_df, x='Age', hue='Survived', kde=True, ax=axes[0,1])
# ... ещё 4 графика
```

Обязательные требования:

- ☑ **Минимум 5 визуализаций** с подписями осей и заголовками
- ☑ **Комментарии** к каждой строке кода
- ☑ После визуализации — Markdown-вывод с наблюдениями (что видно на графиках)
- ☑ **Обосновать** каждую стратегию обработки пропусков (в Markdown)
- ☑ Пути к файлам — **относительные** (`data/train.csv`), чтобы работало из папки модуля

Баллы: 12

---

6 Построение и обучение моделей

Что сделать:

Обучить **минимум 2 модели** для сравнения:

Обязательные модели:

1. **Logistic Regression** (базовая линейная модель)
2. **Decision Tree** (нелинейная модель)

Опционально (для +баллов):

- Random Forest
- Gradient Boosting (XGBoost/LightGBM)

Пайплайн:

```
# 1. Выбор признаков
feature_cols = ['Pclass', 'Sex_Encoded', 'Age', 'SibSp', 'Parch',
                'Fare', 'Family_Size', 'Is_Alone',
                'Embarked_Q', 'Embarked_S']

X = train_df[feature_cols]
y = train_df['Survived']

# 2. Разделение
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Масштабирование (для LR!)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Обучение Logistic Regression
lr_model = LogisticRegression(penalty='l2', C=1.0, max_iter=1000,
random_state=42)
lr_model.fit(X_train_scaled, y_train)

# 5. Обучение Decision Tree
dt_model = DecisionTreeClassifier(max_depth=5, min_samples_split=5,
                                  min_samples_leaf=2, random_state=42)
dt_model.fit(X_train, y_train)  # без масштабирования!
```

Обязательно зафиксировать:

```
import time

start = time.time()
lr_model.fit(X_train_scaled, y_train)
lr_time = time.time() - start
print(f"⏱️ Время обучения LR: {lr_time:.2f} секунд")
```

Markdown-описание моделей:

```
## Модели

| Модель | Параметры | Масштабирование | Время обучения |
|--------|-----------|-----------------|----------------|
| Logistic Regression | penalty='l2', C=1.0 | ☑ Да | X сек |
| Decision Tree | max_depth=5 | ❌ Нет | Y сек |

**Почему эти модели:**
- LR — простая, интерпретируемая, даёт вероятности
- DT — нелинейная, устойчива к выбросам, показывает важность признаков
```

Баллы: 12

7️⃣ Оценка качества

Что сделать:

1. **Рассчитать метрики** для каждой модели:

```
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, roc_auc_score,
                             confusion_matrix, classification_report)

y_pred_lr = lr_model.predict(X_test_scaled)
y_proba_lr = lr_model.predict_proba(X_test_scaled)[:, 1]

print(f"Accuracy:  {accuracy_score(y_test, y_pred_lr):.4f}")
print(f"Precision: {precision_score(y_test, y_pred_lr):.4f}")
print(f"Recall:    {recall_score(y_test, y_pred_lr):.4f}")
print(f"F1-score:  {f1_score(y_test, y_pred_lr):.4f}")
print(f"ROC-AUC:   {roc_auc_score(y_test, y_proba_lr):.4f}")
print("\n", classification_report(y_test, y_pred_lr))
```

2. **Построить матрицу ошибок** (для каждой модели):

```
cm = confusion_matrix(y_test, y_pred_lr)
plt.figure(figsize=(6, 4))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Не выжил', 'Выжил'],
            yticklabels=['Не выжил', 'Выжил'])
plt.xlabel('Предсказано')
plt.ylabel('Истинно')
plt.title('Матрица ошибок: Logistic Regression')
plt.show()
```

3. **Построить ROC-кривые** для обеих моделей на одном графике:

```
fpr_lr, tpr_lr, _ = roc_curve(y_test, y_proba_lr)
fpr_dt, tpr_dt, _ = roc_curve(y_test, y_proba_dt)

plt.plot(fpr_lr, tpr_lr, label=f'LR (AUC = {roc_auc_score(y_test,
y_proba_lr):.3f})', lw=2)
plt.plot(fpr_dt, tpr_dt, label=f'DT (AUC = {roc_auc_score(y_test,
y_proba_dt):.3f})', lw=2)
plt.plot([0, 1], [0, 1], 'k--', label='Случай (AUC = 0.5)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC-кривые')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

4. **Markdown-анализ**:

```
## Оценка качества

| Модель | Accuracy | Precision | Recall | F1 | ROC-AUC |
|--------|----------|-----------|--------|-----|---------|
| Logistic Regression | X | X | X | X | X |
| Decision Tree | Y | Y | Y | Y | Y |

**Анализ:**
- Лучшая модель по ROC-AUC: ...
- DT переобучается сильнее, потому что ...
- Precision выше Recall — модель осторожна в предсказаниях «выживет»
- ...
```

Баллы: 12

---

8 Интерпретация результатов

Что сделать:

1. **Коэффициенты Logistic Regression**:

```
coefficients = pd.DataFrame({
    'Feature': feature_cols,
    'Coefficient': lr_model.coef_[0]
}).sort_values('Coefficient', ascending=False)

plt.figure(figsize=(10, 6))
colors = ['#2ecc71' if c > 0 else '#e74c3c' for c in
coefficients['Coefficient']]
plt.barh(coefficients['Feature'], coefficients['Coefficient'], color=colors)
plt.axvline(0, color='black', linewidth=0.8)
plt.xlabel('Коэффициент (влияние на логарифм шансов)')
plt.title('Важность признаков: Logistic Regression')
plt.show()
```

2. **Feature Importance дерева**:

```
importances = pd.DataFrame({
    'Feature': feature_cols,
    'Importance': dt_model.feature_importances_
}).sort_values('Importance', ascending=False)

sns.barplot(data=importances, x='Importance', y='Feature',
palette='viridis')
plt.title('Важность признаков: Decision Tree')
plt.show()
```

3. **Бизнес-инсайты** (Markdown):

```
## Интерпретация

**Топ-3 признака, повышающих шанс выживания:**
1. Sex_Encoded (женщины) — коэффициент +X
2. Pclass — коэффициент -X (обратная зависимость)
3. Fare — коэффициент +X

**Топ-3 признака, понижающих шанс:**
1. ...

**Бизнес-инсайты:**
- Женщины выживали в 74% случаев, мужчины — 19% (принцип «women and children
first»)
- Пассажиры 1-го класса выживали в 63% случаев, 3-го — 24%
- Одиночки выживали в 30% случаев (нет помощи близких)
- ...
```

Баллы: 8

---

9 Функция предсказания

Что сделать:

Реализовать **функцию** `predict_passenger`, которая принимает данные нового пассажира и возвращает предсказание.

```
def predict_passenger(passenger_data, model, scaler, feature_cols,
use_scaling=True):
    """
    Предсказывает, выживет ли пассажир.

    Параметры:
    ----------
    passenger_data : dict
        Словарь с полями: Pclass, Sex, Age, SibSp, Parch, Fare, Embarked
    model : обученная модель (LR или DT)
    scaler : StandardScaler (для LR) или None (для DT)
    feature_cols : list — список признаков
    use_scaling : bool — нужно ли масштабирование

    Возвращает:
    -----------
    prediction : int — 0 или 1
    probability : float — вероятность выживания
    """
    # 1. Feature Engineering — как при обучении!
    passenger_data['Family_Size'] = passenger_data['SibSp'] +
passenger_data['Parch'] + 1
    passenger_data['Is_Alone'] = int(passenger_data['Family_Size'] == 1)

    # 2. Кодирование
    passenger_data['Sex_Encoded'] = 1 if passenger_data['Sex'] == 'female'
else 0
    passenger_data['Embarked_Q'] = 1 if passenger_data['Embarked'] == 'Q'
else 0
    passenger_data['Embarked_S'] = 1 if passenger_data['Embarked'] == 'S'
else 0

    # 3. DataFrame
    X_new = pd.DataFrame([{
        'Pclass': passenger_data['Pclass'],
```

```
        'Sex_Encoded': passenger_data['Sex_Encoded'],
        'Age': passenger_data['Age'],
        'SibSp': passenger_data['SibSp'],
        'Parch': passenger_data['Parch'],
        'Fare': passenger_data['Fare'],
        'Family_Size': passenger_data['Family_Size'],
        'Is_Alone': passenger_data['Is_Alone'],
        'Embarked_Q': passenger_data['Embarked_Q'],
        'Embarked_S': passenger_data['Embarked_S']
    }])

    # 4. Масштабирование
    if use_scaling:
        X_new = scaler.transform(X_new)

    # 5. Предсказание
    prediction = model.predict(X_new)[0]
    probability = model.predict_proba(X_new)[0, 1]

    return prediction, probability
```

Обязательная демонстрация:

1. **Тестовые примеры** — минимум **5 пассажиров** с разными профилями:

Пример	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Ожидание
1	1	female	25	1	0	100	C	☑ выживет
2	3	male	30	0	0	7.25	S	❌ не выживет
3	2	female	5	3	1	30	S	☑ выживет
4	1	male	60	0	0	200	C	❌ не выживет
5	3	female	22	0	0	8	Q	☑ выживет

2. **Для каждого примера** вывести:
    - Данные пассажира
    - Предсказание LR
    - Предсказание DT
    - Вероятности (в %)
    - Совпадают ли модели
3. **Интерактивный режим** — через `input()` (опционально, для +баллов).

Баллы: 8

🔟 Сохранение модели и вспомогательных объектов

Что сделать:

1. **Сохранить модели** через `joblib`:

```
import joblib

joblib.dump(lr_model, 'models/lr_model.pkl')
joblib.dump(dt_model, 'models/dt_model.pkl')
joblib.dump(scaler, 'models/scaler.pkl')
joblib.dump(le_sex, 'models/le_sex.pkl')
print("☑ Модели и трансформеры сохранены")
```

2. **Сохранить список признаков**:

```
import json

with open('models/feature_cols.json', 'w', encoding='utf-8') as f:
    json.dump(feature_cols, f, ensure_ascii=False, indent=2)
print("☑ Список признаков сохранён")
```

3. **Сохранить метрики**:

```
metrics = {
    'logistic_regression': {
        'accuracy': float(accuracy_score(y_test, y_pred_lr)),
        'precision': float(precision_score(y_test, y_pred_lr)),
        'recall': float(recall_score(y_test, y_pred_lr)),
        'f1': float(f1_score(y_test, y_pred_lr)),
        'roc_auc': float(roc_auc_score(y_test, y_proba_lr)),
        'training_time_sec': float(lr_time)
    },
    'decision_tree': {
        'accuracy': float(accuracy_score(y_test, y_pred_dt)),
        'precision': float(precision_score(y_test, y_pred_dt)),
        'recall': float(recall_score(y_test, y_pred_dt)),
        'f1': float(f1_score(y_test, y_pred_dt)),
        'roc_auc': float(roc_auc_score(y_test, y_proba_dt)),
        'training_time_sec': float(dt_time)
    }
}

with open('models/metrics.json', 'w') as f:
```

```
    json.dump(metrics, f, indent=2)
print("☑ Метрики сохранены")
```

4. **Загрузить файлы в репозиторий**:

```
# Из корня репозитория ml-course-{фамилия}
git add module-1-titanic/models/
git add module-1-titanic/notebook.ipynb
git add module-1-titanic/README.md
git commit -m "Module 1: Add trained models and notebook"
git push origin main
```

Полная структура репозитория после сдачи Модуля 1:

```
ml-course-ivanov/
├── README.md                          ← Общее описание курса
├── .gitignore
└── module-1-titanic/
    ├── README.md                      ← Описание Модуля 1
    ├── notebook.ipynb                 ← Полный отчёт
    ├── data/
    │   ├── train.csv
    │   ├── test.csv
    │   └── titanic_info.md
    ├── models/
    │   ├── lr_model.pkl
    │   ├── dt_model.pkl
    │   ├── scaler.pkl
    │   ├── le_sex.pkl
    │   ├── feature_cols.json
    │   └── metrics.json
    ├── examples/
    │   ├── eda_plots.png
    │   ├── roc_curves.png
    │   └── confusion_matrices.png
    └── requirements.txt
```

Требования:

- ☑ Модели **загружены в репозиторий** (не в Releases!)
- ☑ Файл `feature_cols.json` содержит все признаки **в правильном порядке**
- ☑ В `module-1-titanic/README.md` указаны **метрики** и **дата обучения**
- ☑ В **корневом** `README.md` в таблице модулей для Модуля 1 стоит статус ☑

Баллы: 6

11️⃣ Загрузка модели из репозитория и быстрый инференс

Что сделать:

Продемонстрировать, что модели можно **загрузить из репозитория** и использовать **без переобучения**.

1. **Получить raw-ссылку**:
   - Открыть модель в GitHub → кнопка **Raw** → скопировать URL
   - Формат: `https://raw.githubusercontent.com/username/ml-course-ivanov/main/module-1-titanic/models/lr_model.pkl`
2. **Код загрузки**:

```
import requests
from io import BytesIO
import joblib

# ⚠️ ВАЖНО: путь должен включать папку module-1-titanic/
BASE_URL = "https://raw.githubusercontent.com/username/ml-course-ivanov/main/module-1-titanic"

# Загрузка модели
model_url = f"{BASE_URL}/models/lr_model.pkl"
response = requests.get(model_url)
lr_model_loaded = joblib.load(BytesIO(response.content))
print("☑ Модель загружена из репозитория")

# Загрузка scaler
scaler_url = f"{BASE_URL}/models/scaler.pkl"
scaler_loaded = joblib.load(BytesIO(requests.get(scaler_url).content))
print("☑ Scaler загружен")

# Загрузка признаков
features_url = f"{BASE_URL}/models/feature_cols.json"
feature_cols_loaded = requests.get(features_url).json()
print(f"☑ Загружено {len(feature_cols_loaded)} признаков")
```

3. **Проверка**: предсказать на 3 тестовых пассажирах и сравнить с результатами из ячейки 9 — **результаты должны совпадать**.
4. **Markdown-вывод**:

```
## Быстрый инференс из репозитория
```

```
**Как использовать модель:**
1. Скачать модель по ссылке: [lr_model.pkl](...)
2. Загрузить через `joblib.load()`
3. Вызвать `predict_passenger(data, model, scaler, features)`

**Проверка:** предсказания загруженной модели **совпадают** с исходными.
```

Баллы: 5

1️⃣2️⃣ Выводы

Что сделать:

Markdown-ячейка `## Выводы`, содержащая **3 блока**:

Блок 1: Что получилось ☑

- Достигнутая точность (ROC-AUC на тесте)
- Лучшая модель и её метрики
- Время обучения
- Ключевые инсайты из EDA

Блок 2: Какие были трудности ⚠

- Проблемы с пропусками (Age, Cabin)
- Дисбаланс классов
- Переобучение дерева
- Что не получилось сделать

Блок 3: Возможные улучшения 🚀

- Минимум **5 конкретных идей**:
  - «Извлечь Title из столбца Name (Mr, Mrs, Miss...)»
  - «Добавить взаимодействия признаков (Pclass × Sex)»
  - «Применить SMOTE для балансировки классов»
  - «Попробовать ансамбли (Random Forest, XGBoost)»
  - «Подобрать гиперпараметры через GridSearchCV»
  - «Применить логарифмирование Fare (np.log1p)»

Формат:

```
## Выводы

### ☑ Что получилось
- Достигнут ROC-AUC **0.86** на тестовой выборке
- Лучшая модель: **Logistic Regression** (F1 = 0.78)
- Время обучения: **0.03 секунды**
- Ключевой инсайт: **пол — самый сильный предиктор выживания**

### ⚠ Трудности
```

```
- 20% пропусков в Age пришлось заполнять медианой по группам
- Столбец Cabin (77% пропусков) пришлось удалить
- Дерево переобучалось без ограничения max_depth

### 🚀 Возможные улучшения
1. Извлечь Title из Name — это даст информацию о социальном статусе
2. Применить SMOTE — классы несбалансированы (61.6% vs 38.4%)
3. Попробовать XGBoost/LightGBM
4. ...
```

Баллы: 5

1️⃣3️⃣ Список использованных источников

Что сделать:

Markdown-ячейка `## Источники`. Указать **минимум 5 источников**:

- 📚 Официальная документация scikit-learn / pandas
- 📄 Статья (например, про Titanic dataset)
- 📖 Книга (Géron «Hands-On ML», McKinney «Python for Data Analysis»)
- 🌐 Kaggle-ноутбуки (2-3 качественных)
- 🛠️ Датасет (ссылка на Kaggle)

Формат оформления:

```
## Источники

1. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
2. McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.
3. Kaggle. Titanic - Machine Learning from Disaster. https://www.kaggle.com/c/titanic
4. Scikit-learn Documentation. https://scikit-learn.org/stable/
5. Pandas Documentation. https://pandas.pydata.org/docs/
```

Баллы: 3

1️⃣4️⃣ Приложение

Что сделать:

Финальная секция `## Приложение` с **полным кодом** ноутбука в одном месте.

Требования:

- ☑ **Все комментарии** сохранены
- ☑ Использовать `# %%` для разделения ячеек (формат Jupytext)
- ☑ В начале — список всех зависимостей:

```
# module-1-titanic/requirements.txt
numpy==1.24.0
pandas==2.0.0
matplotlib==3.7.0
seaborn==0.12.0
scikit-learn==1.3.0
joblib==1.3.0
requests==2.31.0
```

Баллы: 3

📊 Критерии оценивания (100 баллов)

Раздел	Баллы	Что проверяется
1. Титульный лист	3	ФИО, группа ПКТб-23-1, рабочая ссылка, путь к модулю
2. Введение	5	Ясность цели, осмысленность задач
3. Теоретическая часть	12	Понимание, свои слова, формулы
4. Описание данных	6	Статистика, ссылка, баланс классов
5. Подготовка данных и EDA	12	Минимум 5 визуализаций, обработка пропусков
6. Построение и обучение	12	2+ модели, время обучения, обоснование
7. Оценка качества	12	Метрики, матрица ошибок, ROC-кривая
8. Интерпретация	8	Важность признаков, бизнес-инсайты
9. Функция предсказания	8	Работающая функция, 5+ примеров

Раздел	Баллы	Что проверяется
10. Сохранение модели	6	Модель в репозитории, структура папки module-1-titanic
11. Загрузка из репо	5	Рабочий код загрузки через BASE_URL + module-1-titanic
12. Выводы	5	Честность, конкретика, 5+ идей
13. Источники	3	5+ источников, оформление
14. Приложение	3	Полный код, комментарии
Бонус	+5	Random Forest / XGBoost / SMOTE / GridSearchCV
Штрафы	−10	Плагиат / нерабочий код / отсутствие модели в репо / неверная структура репозитория

🎯 Шкала оценивания

Баллы	Оценка	Уровень
🟢 **91–100**	**Отлично**	Превосходная работа, все разделы выполнены на высоком уровне, есть бонусные элементы, код работает без ошибок
🟡 **76–90**	**Хорошо**	Все обязательные разделы выполнены, есть незначительные недочёты, модель работает корректно
🟠 **61–75**	**Удовлетворительно**	Основные разделы выполнены, но есть пропущенные пункты, ошибки в коде или слабая интерпретация
🔴 **< 61**	**Не зачтено**	Работа не выполнена или выполнена формально, код не работает, отсутствуют ключевые разделы

Детализация по уровням:

🟢 **Отлично (91–100):**

- Все 14 разделов раскрыты полностью
- Минимум 5 качественных визуализаций + собственные наблюдения
- Модель сохранена в репозитории **в правильной папке** `module-1-titanic/models/` и загружается из raw-ссылки

- Проведена глубокая интерпретация признаков
- Реализованы 5+ примеров предсказаний
- Есть минимум 1 бонусный элемент (Random Forest, XGBoost, SMOTE и т.д.)
- Корневой README.md оформлен с таблицей модулей

🟢 **Хорошо (76–90):**

- Все обязательные разделы присутствуют
- Есть 5 визуализаций, но с минимальным анализом
- Модель в репозитории работает
- Интерпретация поверхностная, но корректная
- Реализовано 3–4 примера предсказаний
- Без бонусов

🟡 **Удовлетворительно (61–75):**

- 1–2 раздела отсутствуют или неполны
- Менее 5 визуализаций или они плохо подписаны
- Модель сохранена, но загрузить её из репо не удалось
- Интерпретация слабая (только коэффициенты без выводов)
- 1–2 примера предсказаний
- Структура репозитория частично нарушена

🔴 **Не зачтено (< 61):**

- Отсутствуют ключевые разделы (EDA, обучение, оценка)
- Код не запускается
- Модель не сохранена / не работает
- Списанная работа или плагиат
- Ноутбук без комментариев и выводов
- Репозиторий не создан или приватный

📌 Требования к оформлению

Jupyter Notebook:

- ☑ **Все ячейки Markdown** — с заголовками `#`, `##`
- ☑ **Все ячейки кода** — с комментариями (минимум 1 комментарий на 2-3 строки)
- ☑ **Выводы** каждой ячейки — текстом, а не только картинкой
- ☑ **Не использовать** `print("Здесь надо что-то вывести")` — все ячейки должны быть выполнены
- ☑ **Порядок ячеек** — строго по структуре отчёта
- ☑ **Пути к файлам** — относительные внутри `module-1-titanic/`

Репозиторий `ml-course-{фамилия}`:

- ☑ Публичный (не приватный!)

- ☑ **Один репозиторий на весь курс** — модули в отдельных папках (`module-1-titanic/`, `module-2-.../`, и т.д.)
- ☑ Корневой `README.md` с таблицей всех модулей
- ☑ `README.md` внутри каждого модуля с описанием
- ☑ `.gitignore` в корне (исключить `__pycache__`, `.ipynb_checkpoints`, `*.csv` > 10 MB)
- ☑ Модели загружены в папку модуля
- ☑ Коммиты информативные (не «update», «fix», «123»), с указанием модуля

Запрещено:

- ❌ Копировать код из интернета **без указания источника**
- ❌ Использовать ChatGPT/Copilot **без раскрытия**
- ❌ Сдавать одну работу на двоих
- ❌ Загружать модели в Releases вместо main-ветки
- ❌ Использовать готовые kernel'ы с Kaggle как есть
- ❌ Создавать **отдельный репозиторий** для каждого модуля
- ❌ Хранить файлы модулей в корне репозитория (без папки `module-N-...`)

🧮 Сроки

Этап	Дедлайн
Создание репозитория `ml-course-{фамилия}`	3 дня
Выбор датасета	3 дня
Черновик (до ячейки 5)	1 неделя
Полный ноутбук	2 недели
Защита (5–7 мин презентация)	2 недели + 1 день

🆘 Как сдавать

1. **Создать репозиторий** на GitHub: `ml-course-{фамилия}` (например, `ml-course-ivanov`)
2. **Создать структуру:**

```
3. mkdir ml-course-ivanov
4. cd ml-course-ivanov
```

```
5. git init
6. mkdir -p module-1-titanic/{data,models,examples}
```

7. **Загрузить файлы Модуля 1** в папку `module-1-titanic/`
8. **Создать корневой** `README.md` с таблицей модулей
9. **Проверить**, что ссылка работает: открыть в режиме инкогнито
10. **Отправить** в LMS (или на почту преподавателя):
    - Ссылку на репозиторий `ml-course-{фамилия}`
    - Прямую ссылку на ноутбук: `https://github.com/username/ml-course-ivanov/blob/main/module-1-titanic/notebook.ipynb`
    - Скриншот метрик из ячейки 7

💡 Пример полного репозитория (после сдачи всех 7 модулей)

```
ml-course-ivanov/
├── README.md
├── .gitignore
├── module-1-titanic/
│   ├── README.md
│   ├── notebook.ipynb
│   ├── data/
│   ├── models/
│   └── requirements.txt
├── module-2-{название}/
│   ├── README.md
│   ├── notebook.ipynb
│   ├── data/
│   ├── models/
│   └── requirements.txt
├── module-3-{название}/
│   └── ...
├── module-4-{название}/
├── module-5-{название}/
├── module-6-{название}/
└── module-7-{название}/
```

Корневой `README.md` **(пример):**

```
# 🎓 ML Course — Портфолио проектов

**Студент:** Иванов Иван Иванович
**Группа:** ПКТб-23-1
```

```
**Дисциплина:** Машинное обучение и ИИ
**Учебный год:** 2025/2026

## 📚 Модули курса

| # | Модуль | Тема | Статус | Ссылка |
|---|--------|------|--------|--------|
| 1 | module-1-titanic | EDA + бинарная классификация | ☑ Готово | [Открыть](./module-1-titanic/) |
| 2 | module-2-regression | Предобработка + регрессия | ⏳ В работе | — |
| 3 | module-3-ensembles | Ансамблевые методы | ⏳ Ожидается | — |
| 4 | module-4-features | Инженерия признаков | ⏳ Ожидается | — |
| 5 | module-5-timeseries | Временные ряды | ⏳ Ожидается | — |
| 6 | module-6-neural | Нейросетевые архитектуры | ⏳ Ожидается | — |
| 7 | module-7-mlops | MLOps и деплой | ⏳ Ожидается | — |

## 📊 Сводная таблица результатов

| Модуль | Лучшая модель | ROC-AUC | Дата сдачи |
|--------|---------------|---------|------------|
| 1 | Logistic Regression | 0.86 | 2026-XX-XX |
| 2 | ... | ... | ... |
```

`module-1-titanic/README.md` (пример):

```
# 🚢 Module 1 — Titanic: EDA + бинарная классификация

**Автор:** Иванов И.И., ПКТб-23-1
**Дата:** 2026-XX-XX

## 📊 Результаты

| Модель | Accuracy | Precision | Recall | F1 | ROC-AUC |
|--------|----------|-----------|--------|-----|---------|
| Logistic Regression | 0.83 | 0.81 | 0.76 | 0.78 | 0.86 |
| Decision Tree | 0.79 | 0.75 | 0.71 | 0.73 | 0.81 |

**Время обучения:** 0.03 сек (LR), 0.01 сек (DT)

## 🚀 Быстрый старт

```python
import joblib
import requests
```
```

```
from io import BytesIO

BASE_URL = "https://raw.githubusercontent.com/username/ml-course-ivanov/main/module-1-titanic"
model = joblib.load(BytesIO(requests.get(f"{BASE_URL}/models/lr_model.pkl").content))
```

🗀 Структура

- `notebook.ipynb` — полный отчёт
- `models/` — модели и метаданные
- `data/` — датасет и описание

```
---

## ❓ Часто задаваемые вопросы

**Q: Можно ли создать отдельный репозиторий для каждого модуля?**
A: ❌ **Нет!** Все модули — в **одном** репозитории `ml-course-{фамилия}`. Каждый модуль — в своей папке (`module-1-titanic/`, `module-2-.../`).

**Q: Можно ли использовать другие модели, кроме LR и DT?**
A: Да, но LR и DT — обязательны для сравнения. За дополнительные модели — бонусные баллы.

**Q: Что если модель показывает ROC-AUC < 0.80?**
A: Это нормально, если обосновано. Главное — показать процесс, а не только результат.

**Q: Обязательно ли использовать Spaceship Titanic?**
A: Нет, Titanic (базовый) — обязательный. Spaceship — для желающих +баллы.

**Q: Можно ли использовать CatBoost/XGBoost?**
A: Да, это будет бонус. Но не заменяет LR и DT.

**Q: Что если в тестовом примере модель ошибается?**
A: Это нормально. Обязательно **проанализируйте**, почему модель ошиблась.

**Q: Обязательно ли делать интерактивный ввод?**
A: Нет, но это +2 бонусных балла.

**Q: Как проверить, что модель в репозитории работает?**
```

A: Скачайте raw-ссылку с путём `module-1-titanic/models/lr_model.pkl`, загрузите модель в отдельной ячейке и предскажите на 3 пассажирах — результат должен совпасть с исходным.

Q: Что делать, если не успеваю к дедлайну?
A: Напишите преподавателю заранее. Штраф за просрочку — 5% от итогового балла за каждый день.

Q: Как быть, если репозиторий уже создан с другим именем?
A: Переименуйте его в `ml-course-{фамилия}` (GitHub → Settings → Repository name → Rename). Ссылка автоматически обновится.

Q: Нужно ли создавать пустые папки `module-2-...`, `module-3-...` заранее?
A: Нет, они создаются по мере прохождения курса. В корневом `README.md` они указаны со статусом «⏳ Ожидается».

Удачи! 🚀 Если что-то непонятно — пишите в чат группы или на почту преподавателя.